

## Practice Dynamic Programming Problems

Although dynamic programming is a simple technique it can be difficult to master. One of the best ways to learn to recognize and solve DP problems is by working through as many examples as possible. Below are listed quite a few problems which have DP solutions (they appear roughly in increasing order of difficulty, in the opinion of your TA, and some of the later ones are actually quite tricky). For each problem, try to identify the “state space” and a recursive formula that relates the optimal solution of a large problem to optimal solutions of smaller subproblems. See if you can identify similarities between some of the problems.

**Example: The Integer Knapsack Problem (Duplicate Items Permitted).** You have  $n$  types of items, where the  $i^{th}$  item type has an integer size  $s_i$  and a real value  $v_i$ . You are trying to fill a knapsack of total capacity  $C$  with a selection of items of maximum value. You can add multiple items of the same type to the knapsack.

*Solution:* Let  $M(j)$  denote the maximum value you can pack into a size  $j$  knapsack. We can express  $M(j)$  recursively in terms of solutions to smaller problems as follows:

$$M(j) = \begin{cases} \max\{M(j-1), \max_{i=1 \dots n} M(j-s_i) + v_i\} & \text{if } j \geq 1 \\ 0 & \text{if } j \leq 0 \end{cases}$$

Computing each  $M(j)$  value will require  $\Theta(n)$  time, and we need to sequentially compute  $C$  such values. Therefore, total running time is  $\Theta(nC)$ . Total space is  $\Theta(C)$ . The value of  $M(C)$  will contain the value of the optimal knapsack packing. We can reconstruct the list of items in the optimal solution by maintaining and following “backpointers” (see the textbook, pages 346 and 354, for more detail).

**1. Maximum Value Contiguous Subsequence.** Given a sequence of  $n$  real numbers  $A_1 \dots A_n$ , determine a contiguous subsequence  $A_i \dots A_j$  for which the sum of elements in the subsequence is maximized.

**2. Making Change.** You are given  $n$  types of coin denominations of values  $v_1 < v_2 < \dots < v_n$  (all integers). Assume  $v_1 = 1$ , so you can always make change for any amount of money  $C$ . Give an algorithm which makes change for an amount of money  $C$  with as few coins as possible.

**3. Longest Increasing Subsequence.** Given a sequence of  $n$  real numbers  $A_1 \dots A_n$ , determine a subsequence (not necessarily contiguous) of maximum length in which the values in the subsequence form a strictly increasing sequence.

**4. Box Stacking.** You are given a set of  $n$  types of rectangular 3-D boxes, where the  $i^{th}$  box has height  $h_i$ , width  $w_i$  and depth  $d_i$  (all real numbers). You want to create a stack of boxes which is as tall as possible, but you can only stack a box on top of another box if the dimensions of the 2-D base of the lower box are each strictly larger than those of the 2-D base of the higher box. Of course, you can rotate a box so that any side functions as its base. It is also allowable to use multiple instances of the same type of box.

**5. Building Bridges.** Consider a 2-D map with a horizontal river passing through its center. There are  $n$  cities on the southern bank with  $x$ -coordinates  $a_1 \dots a_n$  and  $n$  cities on the northern bank with  $x$ -coordinates  $b_1 \dots b_n$ . You want to connect as many north-south pairs of cities as possible with bridges such that no two bridges cross. When connecting cities, you are only allowed to connect the  $i^{th}$  city on the northern bank to the  $i^{th}$  city on the southern bank.

**6. Integer Knapsack Problem (Duplicate Items Forbidden).** This is the same problem as the example above, except here it is forbidden to use more than one instance of each type of item.

**7. Balanced Partition.** You have a set of  $n$  integers each in the range  $0 \dots K$ . Partition these integers into two subsets such that you minimize  $|S_1 - S_2|$ , where  $S_1$  and  $S_2$  denote the sums of the elements in each

of the two subsets.

**8. Edit Distance.** Given two text strings  $A$  of length  $n$  and  $B$  of length  $m$ , you want to transform  $A$  into  $B$  with a minimum number of operations of the following types: *delete* a character from  $A$ , *insert* a character into  $A$ , or *change* some character in  $A$  into a new character. The minimal number of such operations required to transform  $A$  into  $B$  is called the edit distance between  $A$  and  $B$ .

**9. Counting Boolean Parenthesizations.** You are given a boolean expression consisting of a string of the symbols 'true', 'false', 'and', 'or', and 'xor'. Count the number of ways to parenthesize the expression such that it will evaluate to true. For example, there is only 1 way to parenthesize 'true and false xor true' such that it evaluates to true.

**10. Optimal Strategy for a Game.** Consider a row of  $n$  coins of values  $v_1 \dots v_n$ , where  $n$  is even. We play a game against an opponent by alternating turns. In each turn, a player selects either the first or last coin from the row, removes it from the row permanently, and receives the value of the coin. Determine the maximum possible amount of money we can definitely win if we move first.

**11. Two-Person Traversal of a Sequence of Cities.** You are given an ordered sequence of  $n$  cities, and the distances between every pair of cities. You must partition the cities into two subsequences (not necessarily contiguous) such that person A visits all cities in the first subsequence (in order), person B visits all cities in the second subsequence (in order), and such that the sum of the total distances travelled by A and B is minimized. Assume that person A and person B start initially at the first city in their respective subsequences.

**12. Bin Packing (Simplified Version).** You have  $n_1$  items of size  $s_1$ ,  $n_2$  items of size  $s_2$ , and  $n_3$  items of size  $s_3$ . You'd like to pack all of these items into bins each of capacity  $C$ , such that the total number of bins used is minimized.